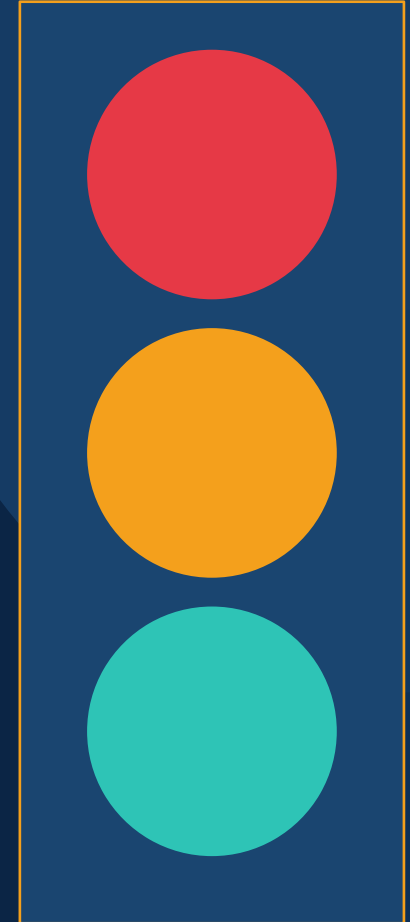


Traffic Light Controller

Configurable AXI-Based Design on PYNQ-Z2 Board

Vivado Block Design · Verilog FSM · XDC Constraints · Jupyter / Python



System Architecture

How hardware & software layers work together on the PYNQ-Z2 board

HARDWARE LAYER

Vivado Block Design

ZYNQ7 PS + Processor System Reset block + custom traffic_fsm RTL, clocked by FCLK_CLK0.

Verilog FSM

Moore FSM with RED→YELLOW→GREEN states. Counter register (50M cycles) governs timing.

XDC Constraints

Maps led[3:0] outputs to pins R14/P14/N16/M14 using LVCMOS33 I/O standard.



SOFTWARE LAYER

Bitstream Generation

Vivado synthesises and implements the design, producing a .bit file for FPGA programming.

PYNQ Overlay

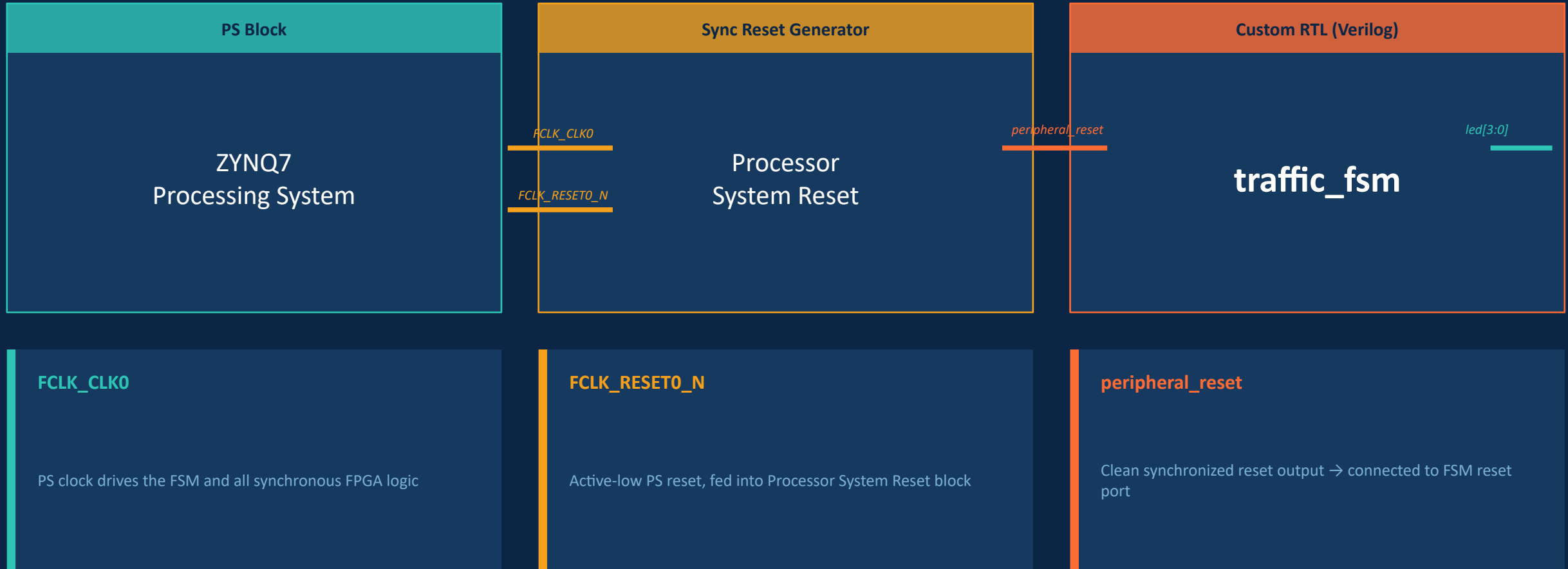
Python Overlay class loads the bitstream: `ol = Overlay('design_1.bit')`. FSM starts instantly.

ipywidgets Dashboard

RED/YELLOW/GREEN indicators with live state label, countdown timer and cycle counter — visualization only.

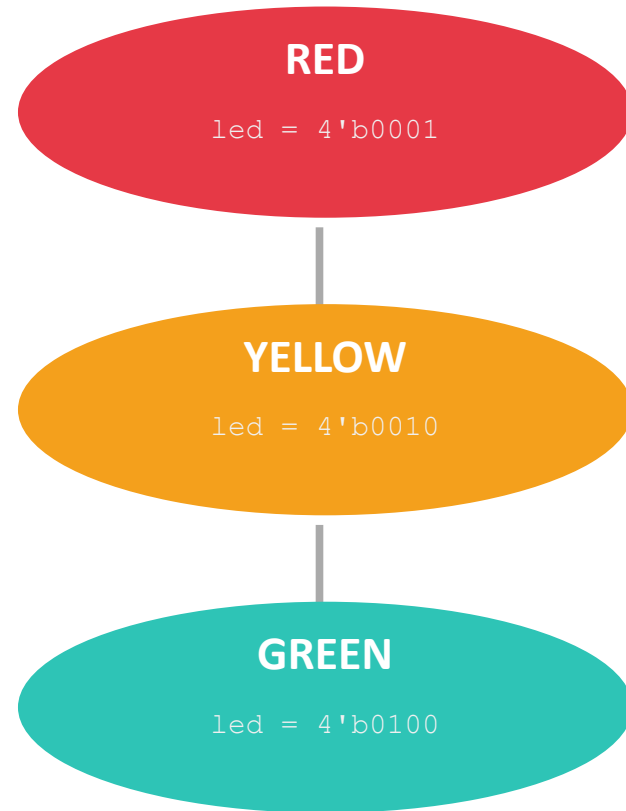
Vivado Block Design

ZYNQ PS · Processor System Reset · Custom traffic_fsm RTL Module



Verilog FSM Design

Moore Machine • 3 States • 50 Million Clock Cycle Delay Per State



50M cycles
per state



Ports

```
clk - input clock  
reset - input reset  
led[3:0] - output
```



Registers

```
state - FSM state  
counter - delay timer
```



Timing

```
counter++ each edge  
Reset at 50,000,000  
→ triggers next state
```



Output

```
Moore: out = f(state)  
RED=0001 YLW=0010  
GRN=0100 1 LED at once
```

XDC Constraints & Output Mapping

Physical Pin Mapping · LVCMOS33 Voltage Standard · DRC Configuration

Signal	FPGA Pin	Onboard LED	Standard
led_0[0]	R14	LED 0 (LD0)	LVCMOS33
led_0[1]	P14	LED 1 (LD1)	LVCMOS33
led_0[2]	N16	LED 2 (LD2)	LVCMOS33
led_0[3]	M14	LED 3 (LD3)	LVCMOS33

LED Output States

 RED → 0001

 YELLOW → 0010

 GREEN → 0100

LVCMOS33

Sets correct 3.3V I/O logic levels for all LED output pins on PYNQ-Z2.

Pin Mapping

Directly connects FSM logical outputs to the four physical onboard LEDs.

DRC Config

Severity of certain DRC rule violations reduced to avoid synthesis warnings.

End-to-End Workflow

Complete pipeline: Vivado hardware design → FPGA bitstream → Python visualization



Understanding PS & PL in Zynq 7020

PS — Processing System

- ▶ Dual-core ARM Cortex-A9 @ 667MHz
- ▶ Runs Linux OS (PYNQ Image)
- ▶ Hosts Jupyter Notebook Server
- ▶ Executes Python code & overlays
- ▶ Controls peripherals via AXI bus
- ▶ 512MB DDR3 RAM available
- ▶ Handles USB, Ethernet, UART

AXI

PL — Programmable Logic

- ▶ Artix-7 FPGA Fabric (28nm)
- ▶ Runs hardware bitstream (.bit file)
- ▶ Executes Traffic Light FSM
- ▶ Operates at 50 MHz FCLK_CLK0
- ▶ Controls LD0/LD1/LD2 directly
- ▶ 17,600 LUTs + 35,200 flip-flops
- ▶ Loaded via Overlay() in Python

Challenges & Errors Faced



Custom IP Attempt via Vitis → Vivado

Initially tried creating a Custom IP block in Vitis HLS and importing it into Vivado's IP catalog. After packaging and adding to the block design, the PYNQ board threw runtime errors — the overlay could not find the expected IP hierarchy, causing 'AttributeError: Could not find IP or hierarchy gpio in overlay'.



DRC Errors

NSTD-1: Unspecified I/O Standard on 7/137 ports. UCIO-1: Unconstrained logical ports for led_0[3:0] and gpio_rtl_0.



FSM Clock Unconnected

FCLK_CLK0 was not wired to traffic_fsm_0 clk pin — FSM never counted, LED stuck permanently on RED state.



AXI SmartConnect

Failed to deliver smartconnect_v1_0 update contents. IP was outdated and caused Block Design compilation failure.



Syntax Error

HDL-9-1206: Syntax error in traffic_fsm.v at line 27 due to incorrect parameter usage — caused synthesis failure.

Solutions & Fixes Applied

01

Custom IP Failure

Abandoned Custom IP approach. Switched to direct RTL (Verilog) FSM module in Block Design — simpler and more reliable for PYNQ.

02

AXI GPIO Conflict

Deleted AXI GPIO, AXI SmartConnect, and Discontinued Constant blocks from Block Design — removed all unnecessary IP.

03

Clock Not Connected

Used Tcl Console command to explicitly connect FCLK_CLK0 to traffic_fsm_0/clk pin ensuring FSM gets 50MHz clock.

04

Constraints Wrong Port Names

Updated XDC from GPIO_0_tri_o[] to correct led_0[] port names matching the actual Block Design external port.

05

FSM Stuck on RED

Fixed counter increment logic placement — moved inside else block ensuring counter increments every clock cycle correctly.

Jupyter Notebook

PYNQ Overlay · ipywidgets Dashboard · Visualization Only

STEP 1 — LOAD BITSTREAM

```
from pynq import Overlay  
ol = Overlay("design_1.bit")
```

02

Create Dashboard

ipywidgets — 3 indicator buttons (RED/YELLOW/GREEN) + status label + countdown timer + cycle counter arranged in a VBox layout.

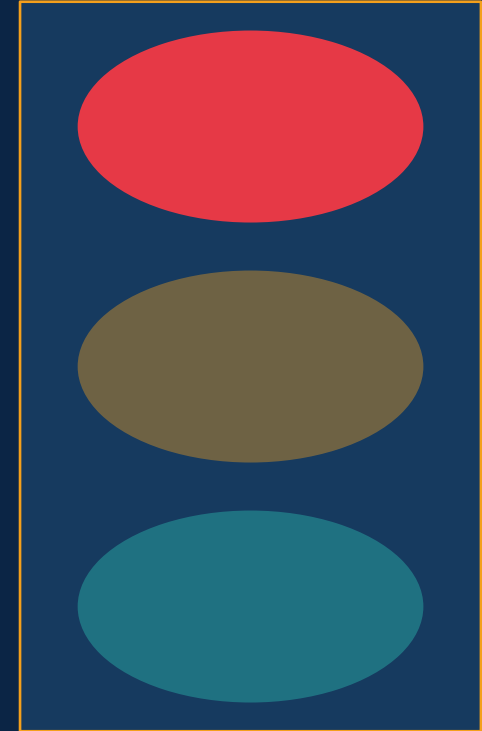
03

Run Cyclic Loop

Loop: RED→YELLOW→GREEN, each state ~2 sec. Highlighted button updates. Status label, timer and cycle count refresh each iteration.

Note: Dashboard is visualization only — actual LED switching is performed entirely by the FSM in FPGA hardware.

Dashboard Preview



State: RED
Timer: 1.8s
Cycles: 3

Project Conclusion

- Configurable AXI-based Traffic Light FSM implemented on ZYNQ PYNQ-Z2
- Moore state machine: RED→YELLOW→GREEN via 50M-cycle counter timing
- XDC constraints map led[3:0] to physical pins R14, P14, N16, M14
- Bitstream loaded onto FPGA via Python PYNQ Overlay class
- ipywidgets dashboard visualizes state, timer and cycle count — hardware-independent
- Working output verified: onboard LEDs blink per the designed traffic sequence

Thank You!

